

Relatório de evolução -- Sprint 4

Chatbot ChargeGrid Intelligence -- RAG medido, confiável e utilizável EV Challenge -- GoodWe / FIAP · Prompt and Artificial Intelligence · 2026.2 Pedro Sampaio Mochnacs Arruda · RM 573522 · Turma 1CCPG

1. Resumo da evolução

Nas Sprints 1 e 2 o chatbot respondia com base em 22 frases soltas da planilha SP2, encontradas por palavra-chave, sem dizer de onde vinha cada informação. Na Sprint 3 o núcleo foi reescrito em LangChain, com memória por tokens, saída estruturada e cinco camadas de segurança, mas a busca continuou a mesma.

Na Sprint 4 a busca virou um RAG de verdade: dez documentos do domínio (manual do carregador GoodWe HCA, material do Secovi-SP sobre condomínios, guias da Arval e do PROMOB-e, e os documentos do próprio ChargeGrid) são divididos em chunks, vetorizados com nomic-embed-text e guardados num ChromaDB persistente. Toda resposta cita documento e seção, pergunta sem resposta na base é recusada em código, e instrução escondida dentro de documento é removida antes de chegar ao modelo. O resultado é medido com RAGAS em três iterações e aparece numa interface web (Gradio) com as fontes visíveis.

2. Pipeline RAG

```
PyMuPDFLoader / Markdown -> RecursiveCharacterTextSplitter -> nomic-embed-text -> ChromaDB
pergunta -> (reescrita se encadeada) -> busca com filtro de acesso e limiar -> blindagem
-> prompt RAG versionado -> modelo -> guarda de saída -> citação garantida
```

Base de conhecimento. 5 PDFs públicos (GoodWe, Secovi-SP x2, Arval, PROMOB-e) e 5 documentos escritos para o projeto (manual de operação e tabela tarifária do ChargeGrid, FAQ de recarga, modelo de regimento de condomínio e o relatório histórico SP2, gerado da planilha real). Cada documento tem metadados de tipo, nível de acesso e link de origem; o relatório SP2 é acesso: gestao e só entra na busca para o perfil de gestão.

Chunking. Comparamos duas estratégias:

- `fixo_1000` -- 1000 caracteres, overlap 150, sobre o texto cru da página.
- `secao_500` -- 500 caracteres, overlap 75, dentro de cada página ou seção, com o título do documento e a seção no início de cada chunk.

A iteração 1 usa `fixo_1000` e as iterações 2 e 3, `secao_500`; a escolha final segue a recuperação (`hit@k`) e o RAGAS da tabela da seção 3. O argumento a favor do chunk menor: uma página do manual GoodWe mistura instalação, LEDs e app, e um chunk de 1000 caracteres carrega tudo isso junto, diluindo a similaridade; o cabeçalho "documento -- seção" ajuda a busca e dá a citação pronta. O custo é ter mais chunks (493 contra 295). O overlap de 15% faz a frase cortada no limite aparecer inteira num dos vizinhos.

Parâmetros. `k = 4` trechos, limiar de relevância 0,45 (abaixo disso o trecho é descartado; calibrado por `evals/calibrar_limiar.py`), temperatura 0. Detalhes e comparação de modelos em `docs/relatorio_modelos.md`.

3. Comparativo antes/depois

Mesmo eval set (`evals/eval_set_rag.json`, 24 casos: 16 com resposta nos documentos, 2 de gestão, 2 perguntas encadeadas, 2 sem resposta na base, 1 fora de escopo e 1 de dado restrito), mesmo juiz RAGAS para as duas versões.

Critério	Sprints 1/2 (versão original)	Sprint 04 (RAG avaliado)
Recuperação	palavra-chave em 22 frases da planilha SP2	busca vetorial (nomic-embed-text + ChromaDB) em 10 documentos, com filtro de acesso e limiar
Faithfulness (RAGAS)	pendente	pendente

Critério	Sprints 1/2 (versão original)	Sprint 04 (RAG avaliado)
Faithfulness por iteração	versão única	it1 pendente -> it2 pendente -> it3 pendente
Answer relevancy (RAGAS)	pendente	pendente
Answer relevancy por iteração	versão única	it1 pendente -> it2 pendente -> it3 pendente
Qualidade do contexto recuperado (documento certo entre os trechos)	pendente	pendente
Presença de citação de fonte	pendente	pendente
Recusa correta (sem resposta na base, fora de escopo, dado restrito)	pendente	pendente
Checagens determinísticas OK	pendente	pendente
Latência média por turno	pendente	pendente
Tokens por turno (média)	pendente	pendente

Iterações do RAG (o ganho de cada uma é atribuído à mudança listada):

- Iteração 1: chunk fixo 1000/150, prompt rag_v1, k=3, sem guardas em código.
- Iteração 2: chunk por seção 500/75 com cabeçalho, prompt rag_v2 (grounding + citação), limiar 0,45, recusa em código.
- Iteração 3: prompt rag_v3 com exemplos + reescrita da pergunta encadeada.

Iteração	Faithfulness	Ganho	Answer relevancy	Ganho	Recuperação (hit@k)	Citação	Modelo citou [n]	Recusa correta
Iteração 1	pendente	--	pendente	--	pendente	pendente	pendente	pendente
Iteração 2	pendente	--	pendente	--	pendente	pendente	pendente	pendente
Iteração 3	pendente	--	pendente	--	pendente	pendente	pendente	pendente

Segurança:

Teste	Resultado
Injeção via documento	pendente
Bateria da Sprint 3 (24 casos, 12 ataques)	pendente

Os números saem de evals/resultados/*.json e são reproduzidos com `python -m evals.run_legacy_rag`, `python -m evals.run_iteracoes`, `python -m evals.seguranca_documentos` e `python -m evals.run_evais`.

4. Problemas encontrados e soluções

PDF diagramado saía uma palavra por linha. O guia da Arval é um infográfico: o PyMuPDF devolvia cada palavra numa linha separada ("Os", "veículos", "elétricos"...), e o splitter cortava nessas quebras, gerando chunks sem sentido. Passamos a juntar quebras simples de linha nos PDFs (linha em branco continua separando parágrafo). A apresentação do Secovi de 2026 é quase toda imagem (4,7 mil caracteres em 40 páginas); mantivemos na base, mas ela pouco contribui -- OCR ficou como próximo passo.

A blindagem de documento barrava texto normal. A primeira versão reaproveitava os padrões de ataque da Sprint 3 e marcou "o Secovi acompanhará as novas diretrizes" como instrução injetada. Documento técnico fala de regra, limite e acesso o tempo todo, então escrevemos padrões próprios, que só pegam frase dirigida ao assistente ("nova instrução:", "o assistente deve...", "[SYSTEM]"). O teste `test_base_real_sem_falso_positivo` roda a blindagem nos 493 chunks reais e exige zero remoções.

Pergunta encadeada não achava nada. Limitação anotada na Sprint 3: "e no horário de ponta?" não tem termo para buscar. Com a recusa em código da Sprint 4 isso piorava, porque a busca vazia virava "não encontrei". Agora, quando há histórico e a pergunta é curta ou começa com "e", "esse", "dele"..., o modelo reescreve a pergunta completa antes da busca (iteração 3).

Relatório comercial podia vazar para o motorista. Na Sprint 3 a fronteira de acesso era um if na montagem do contexto. Com tudo num banco vetorial só, um trecho do relatório SP2 poderia aparecer para qualquer pergunta parecida. A solução foi o metadado acesso em cada documento e o filtro where na busca do Chroma: sem perfil de gestão, o relatório nem entra no ranking.

5. Autoria e divisão de trabalho

Trabalho individual, com autorização do professor.

Nome	RM	Turma	Tarefas
Pedro Sampaio Mochnacs Arruda	573522	1CCPG	Todas: base de conhecimento, pipeline RAG, segurança, avaliação RAGAS, interface web e relatórios